SOSNOSDP

Matrix sum-of-squares programs in SOSTOOLS,
solved without semidefinite programming

User manual, version 0.1.0

Corentin Briat*

October 2026

Abstract

SOSNOSDP is a MATLAB package (also running in GNU Octave) that solves sum-of-squares (SOS) programs written with SOSTOOLS without any semidefinite programming (SDP) solver. Constraint polynomials, scalar or matrix-valued, are represented by their values at interpolation nodes, and the program is solved by a non-symmetric interior-point method that only evaluates the barrier of the dual cone. This is the approach of Papp and Yıldız [11], in its matrix-valued form due to Kapelevich, Coey and Vielma [6] and implemented in the Julia solver Hypatia [4]. SOSNOSDP re-implements the matrix weighted SOS cone and the interior-point method in MATLAB, connects them to SOSTOOLS [16], adds interval constraints that need no multipliers, and certifies every returned solution. This manual explains how the package works and how to use it.

Contents

1 Introduction **2**

2 Installation **3**

3 Quick start **3**

4 How it works **4**

4.1 SOS programs and the semidefinite route 4
4.2 Interpolant representation and the matrix WSOS cone 4
4.3 The interior-point method 4
4.4 Certificates 5
4.5 From a SOSTOOLS program to cones 5
4.6 Size compared with the SDP 6

5 Reference **6**

5.1 Functions 6
5.2 Options of `sossolve_nosdp` 7
5.3 Statuses and output 7

*www.briat.info; corentin@briat.info

6 Good practice and numerical limits **7**

7 Examples **7**

7.1 Example 1: polynomial minimization 8

7.2 Example 2: matrix SOS 8

7.3 Example 3: sampled-data stabilization by clock-dependent LMIs 8

7.4 Example 4: largest sampling interval under aperiodic sampling 8

7.5 Example 5: stochastic sampled-data systems 9

8 Verification **9**

9 Relation to other software **9**

10 Limitations and future work **10**

11 How to cite **10**

1 Introduction

SOSTOOLS [16] converts an SOS program into an SDP by writing every SOS polynomial with a positive semidefinite Gram matrix, and passes the SDP to a solver such as SeDuMi [17]. The Gram matrices of a polynomial of degree $2d$ in one variable have about $d^2/2$ entries, while the polynomial has only $2d+1$ coefficients, and for an $m \times m$ matrix polynomial the Gram matrix has side $m(d+1)$. The lifting therefore dominates time and memory at high degree.

Papp and Yıldız [11] showed that the lifting can be avoided entirely. A polynomial is stored by its values at unisolvent interpolation nodes, the SOS cone appears as the dual slack of a conic program, and the homogeneous interior-point method of Skajaa and Ye [14, 10] needs a barrier only for the dual of the SOS cone. That barrier, $-\log\det\Lambda(x)$, has cheap closed-form derivatives in the interpolant basis. Their MATLAB solver alfonso [12] implements the scalar case. The extension to SOS matrices, using the SOS cones in general algebras of Papp and Alizadeh [9], is due to Kapelevich, Coey and Vielma [6], and is implemented in Hypatia [4, 5].

SOSNOSDP brings this approach to SOSTOOLS users. Its contributions are practical:

- a MATLAB implementation of the matrix weighted SOS cone of [6] and of the homogeneous interior-point method;
- `sossolve_nosdp`, a replacement for `sossolve` that reads a SOSTOOLS program and solves it without SDP, after which `sosgetsol` works unchanged;
- native interval constraints for scalar and matrix polynomials, without multipliers, and a helper for normalized time variables;
- certificates for the coefficients actually returned, with explicit checks against rounding errors.

The theory of the cone, its barrier and the recovery of SOS decompositions is that of [11, 6]. Please cite those works together with this package (Section 11).

2 Installation

Requirements. MATLAB R2016b or later, or GNU Octave 8, and SOSTOOLS 4 (tested with the version on GitHub, https://github.com/oxfordcontrol/SOSTOOLS, October 2026). Programs must be written in SOSTOOLS' default polynomial mode (`pvar`). No SDP solver is needed.

MATLAB.

```
addpath(genpath('/path/to/SOSTOOLS'));
addpath('/path/to/sosnosdp');
setup_nosdp
```

`setup_nosdp` adds the folders `core`, `bridge` and `examples` to the path. SOSTOOLS refuses to build a program unless an SDP solver is on the path. If none is installed, `setup_nosdp` adds a placeholder `sedumi.m` that only raises an error when called. It is not added when a real solver is present.

GNU Octave. SOSTOOLS 4 relies on three MATLAB behaviours that Octave lacks (sparse logical indexing and broadcasting of sparse arrays). The script `octave/patch_sostools.py` applies three one-line edits to a copy of SOSTOOLS, which change nothing under MATLAB:

```
git clone https://github.com/oxfordcontrol/SOSTOOLS.git
python3 sosnosdp/octave/patch_sostools.py SOSTOOLS
octave-cli --eval "SOSTOOLS_DIR='SOSTOOLS'; run('sosnosdp/octave/run_tests_octave.m')"
```

Two limitations of SOSTOOLS remain in Octave: products of a polynomial by a `dpvar` fail (write the `dpvar` first, for example `s*(x*(2-x))`), and the default form of `sosmatrixineq` fails. See `octave/README_OCTAVE.md`.

3 Quick start

Largest a such that $M(x) = \begin{bmatrix} 1+x^2 & ax \\ ax & 2+x^2 \end{bmatrix}$ is positive semidefinite for all x. The exact answer is $1+\sqrt{2}$.

```
pvar x a

M = [1 + x^2, a*x;
     a*x,     2 + x^2];

prog = sosprogram(x, a);
prog = sosmatrixineq(prog, M);      % matrix SOS constraint
prog = sossetobj(prog, -a);         % maximize a

[prog, info] = sossolve_nosdp(prog);

double(sosgetsol(prog, a))          % 2.41421343
info.status                         % 'optimal'
```

The only change with respect to a SOSTOOLS script is the call to `sossolve_nosdp`.

4 How it works

4.1 SOS programs and the semidefinite route

A symmetric $r \times r$ matrix polynomial P of degree $2d$ is a sum of squares if $P = \sum_l Q_l^\top Q_l$ for polynomial matrices Q_l. With a basis b of the polynomials of degree at most d this holds if and only if $P(x) = (b(x) \otimes I_r)^\top S\,(b(x) \otimes I_r)$ for some $S \succeq 0$ [8, 13]. Weighted SOS (WSOS) polynomials $\sum_j g_j \sigma_j$, with σ_j SOS and weights $g_j \geq 0$ on a set D, certify positivity on D [13]. An SOS program asks for decision variables z such that affine polynomial expressions in z are (W)SOS. SOSTOOLS introduces one Gram matrix S per constraint and solves the resulting SDP.

4.2 Interpolant representation and the matrix WSOS cone

Following [7, 11], a polynomial of degree at most $2d$ is stored by its values at N nodes $\xi_1, \ldots, \xi_N$ that are unisolvent, meaning that only the zero polynomial vanishes at all of them. For an $r \times r$ matrix polynomial this is the vector $(\operatorname{svec} P(\xi_1), \ldots, \operatorname{svec} P(\xi_N))$, where svec stacks the upper triangle with off-diagonal entries scaled by $\sqrt{2}$.

Let $\tilde{p}_{ji} = \sqrt{g_j(\xi_i)}\, b_j(\xi_i)$ be the weighted basis vectors at the nodes. The matrix WSOS cone and its dual are [6, 9]

$$K = \Big\{ \big(\textstyle\sum_j (\tilde{p}_{ji} \otimes I)^\top \Phi_j (\tilde{p}_{ji} \otimes I) \big)_{i=1}^{N} : \ \Phi_j \succeq 0 \Big\},$$
$$K^* = \Big\{ W : \ \Lambda_j(W) = \textstyle\sum_i \tilde{p}_{ji} \tilde{p}_{ji}^\top \otimes W_i \succeq 0 \ \ \forall j \Big\}.$$

Membership in K^* is a Cholesky test, and $F(W) = -\sum_j \log\det \Lambda_j(W)$ is a logarithmically homogeneous self-concordant barrier for K^* with parameter $r \sum_j L_j$, where L_j is the length of b_j. With $M_{ik}^{(j)} = (\tilde{p}_{ji} \otimes I)^\top \Lambda_j(W)^{-1} (\tilde{p}_{jk} \otimes I)$,

$$\nabla F(W)_i = -\sum_j M_{ii}^{(j)}, \qquad \nabla^2 F(W)[H, H'] = \sum_j \sum_{i,k} \operatorname{tr}\big(M_{ki}^{(j)} H_i M_{ik}^{(j)} H'_k \big).$$

For $r = 1$ these are the formulas of [11], and for general r those of [6, Sec. 5.1]. The PSD cone is the special case $N = 1$, $\tilde{p} = 1$. These formulas are implemented in `core/nsd_cone_eval.m`. The bases $\tilde{P}_j$ are orthonormalized, as recommended in [11, Sec. 4.1].

4.3 The interior-point method

After the translation of Section 4.5, every program has the form

$$\max_z \ b^\top z \quad \text{s.t.} \quad c + Lz \in \mathcal{K} = K_1 \times \cdots \times K_p,$$

which is the dual of $\min c^\top x$ s.t. $-L^\top x = b$, $x \in \mathcal{K}^*$. The homogeneous self-dual embedding of this pair is solved by the predictor-corrector method of Skajaa and Ye [14], as in [11, 10]. It only needs the barrier F of $\mathcal{K}^*$, and the SOS cone $\mathcal{K}$ appears only through the dual slack. The Newton systems are solved through a Schur complement of size equal to the number of decision variables (`core/nsd_ipm.m`). The implementation uses practical neighbourhood parameters (predictor to the boundary of a neighbourhood of size 0.7, up to six correctors back to 0.3) rather than the theoretical ones of [10], so the worst-case iteration bound is not claimed. Correctness rests on the certificates of Section 4.4.

4.4 Certificates

The method approaches $\mathcal{K}$ from outside, through the dual iterates, and $\mathcal{K}$ has no direct membership test. Every returned solution is therefore certified (`core/nsd_certify_point.m`, `core/nsd_check_values.m`).

1. *Projection.* At an iterate with $\delta = \mu/\tau$, the decision variables are re-solved as $z^\star = \arg\min_z \|c + Lz + \delta\nabla F(x)\|_{\nabla^2 F(x)^{-1}}$, so that $S = c + Lz^\star$ lies as close as possible to the central point $\hat{s} = -\delta\nabla F(x)$.
2. *Dikin test.* If $r = \|S - \hat{s}\|_{\nabla^2 F(x)^{-1}}/\delta < 1$, then S lies in the Dikin ellipsoid of the conjugate barrier at $\hat{s}$, hence in the interior of $\mathcal{K}$. Moreover $S - (1-r)\hat{s} \in \mathcal{K}$ and $\hat{s} - \delta\lambda_{\max}(\Lambda(x))^{-1}E \in \mathcal{K}$ for the fixed interior element $E = \mathcal{A}(I)$, which gives a scale-free depth $(1-r)\,\delta\,\|E\|/(\lambda_{\max}\|S\|)$. Because the inverse Hessian is ill-conditioned near the boundary, the test is accepted only if this depth is at least `depth_tol` (default 10^{-8}).
3. *Gram test.* Otherwise the explicit Gram matrices $\Phi_j = \Lambda_j(x)^{-1}\Lambda_j(w)\Lambda_j(x)^{-1}$, $w = \nabla^2 F(x)^{-1}S$, are formed. This is the recovery of [11, Thm. 5.1] in matrix form. They must be positive definite with $\lambda_{\min} > 10^{-12}\lambda_{\max}$ and reproduce S with relative residual below 10^{-10}. This check does not depend on the accuracy of the Hessian. The Gram matrices are used only here, never in the solve.
4. *Returned coefficients.* The certificate is evaluated for the values $c + Lz$ of the coefficients actually returned to `sosgetsol`, not only for internal quantities.

For infeasible programs the method returns $x \in \mathcal{K}^*$ with $c^\top x < 0$. Then no z with $\|z\| < |c^\top x|/\|L^\top x\|$ is feasible (`info.farkas_radius`). This is a global proof of infeasibility only when the radius is large compared with the expected size of solutions.

4.5 From a SOSTOOLS program to cones

SOSTOOLS stores every constraint as polynomial coefficients that are affine in the decision vector u, namely $b - A^\top u$ over a list of monomials (fields `prog.expr.At`, `b`, `Z`). The Gram matrices are created only inside `sossolve`. `sossolve_nosdp` reads the program at that point (`bridge/sossolve_nosdp.m`):

1. *Decision variables.* Free variables (`sosdecvar`, `sospolyvar`, `sospolymatrixvar`) are kept. Gram blocks that SOSTOOLS creates for `sossosvar`, `sossosmatrixvar` and `sosposmatr` are parametrized by their symmetric part and constrained to the PSD cone of the same method. `sossosvar_nosdp` avoids these Gram blocks altogether.
2. *Global SOS constraints* (`sosineq`, `sosmatrixineq`). The monomials of the Gram basis are taken from the Newton box of the polynomial. Nodes unisolvent for their products are approximate Fekete points, selected by column-pivoted QR from Chebyshev points (one variable) or Halton points (several variables) [15, 11]. Monomials that cannot occur in an SOS polynomial become equality constraints. The default form of `sosmatrixineq`, $y^\top M(x)y$ with auxiliary variables y, is converted back to the matrix $M(x)$, so that the matrix cone of [6] is used rather than a scalar cone in (x, y).
3. *Interval constraints.* For `sosineq_interval` and `sosmatrixineq_interval` on $[a, b]$ the cone has weights 1 and $(t-a)(b-t)$, Chebyshev–Lobatto nodes and orthonormalized Chebyshev bases. Half-line constraints of SOSTOOLS (`sosineq(prog,p,[a inf])`) use weights 1 and $t - a$. No multiplier is introduced.
4. *Equalities* (`soseq`) are eliminated through a null-space basis.

5. *Reparametrization.* The constraint matrix is factored as $L\Pi = QR$ with column pivoting, and the solver works with the orthonormal coordinates Q. The problem depends only on the range of L, so this is exact. It makes the method independent of the user's basis and scaling, and removes redundant variables. Numerically dependent directions are reported (`info.near_dependent`).

6. *Write-back.* The SOSTOOLS decision vector is reconstructed and stored in the field `prog.solinfo.RRx`, which is what `sosgetsol` reads.

4.6 Size compared with the SDP

For an $r \times r$ constraint of degree $2d$ in one variable with an interval weight, the interpolant cone has dimension $(2d+1)r(r+1)/2$ and barrier parameter $r(2d+1)$. The corresponding SDP has the same barrier parameter and as many equality constraints as the cone has entries, but in addition Gram unknowns of order r^2d^2. One iteration costs $O(r^6d^3)$ operations here, against $O(r^6d^6)$ for a general SDP interior-point method [11, 6].

5 Reference

5.1 Functions

`[prog, info] = sossolve_nosdp(prog, opts)`
Solves a SOSTOOLS program without SDP. Options and outputs are listed below.

`prog = sosineq_interval(prog, p, t, [a b])`
$p(t) \geq 0$ for $t \in [a, b]$. p may depend on decision variables and on the program variable t only.

`prog = sosmatrixineq_interval(prog, M, t, [a b])`
$M(t) \succeq 0$ for $t \in [a, b]$, M square and symmetric.

`[prog, s] = sossosvar_nosdp(prog, Z)`
SOS decision polynomial in the monomial basis Z, represented by its coefficients and its own SOS cone. The program remains solvable by `sossolve`.

`[s, tm] = nsd_timevar('s', [a b])`
Normalized variable $s \in [-1, 1]$ for a physical interval, with `tm.ddt(Y)` (dY/dt), `tm.at(Y, t0)` (Y at time t_0), `tm.interval(I)` (an interval in s) and `tm.t(s0)`.

`setup_nosdp`
Adds the package to the path.

`nsd_cone`, `nsd_solve`
Low-level interface: `K = nsd_cone(r, Ps)` builds a cone from the weighted basis matrices, and `out = nsd_solve(cones, L, c, b, opts)` solves $\max b^\top z$ s.t. $c + Lz \in \mathcal{K}$ without SOSTOOLS.

5.2 Options of `sossolve_nosdp`

Option	Default	Meaning
`certificate`	`'auto'`	`'auto'`: Dikin test, Gram check where inconclusive; `'dikin'`; `'gram'`
`depth_tol`	10^{-8}	depth a Dikin certificate must prove
`max_iter`, `tol`	200, 10^{-8}	iterations and tolerance of the interior-point method
`box`	`[-1 1]`	box in which nodes of global SOS constraints are chosen
`rank_tol`	10^{-14}	tolerance for removing dependent decision directions
`verbose`	`false`	print the iterations

5.3 Statuses and output

`feasible` feasibility program, certified solution.

`optimal` the method converged; the returned solution is the last certified iterate, strictly feasible and slightly below the optimum (for SOS programs the optimum lies on the boundary of the cone).

`infeasible` a Farkas vector was found; see `info.farkas_radius`.

`uncertified_*` no certificate could be established; the last iterate is still returned so that it can be checked by other means.

Further fields of `info`: `iters`, `time`, `obj`, `cones` (one description per cone), `cert_method` and `depth` (per cone), `dikin_ratio`, `rank`, `near_dependent`, `barrier_parameter`, `accuracy` (`full` or `reduced`), `last_failed_certificate`.

6 Good practice and numerical limits

Normalize interval variables. SOSTOOLS stores all polynomials in monomial coefficients of the declared variable. On a wide interval high-degree monomials are numerically dependent: for a clock-dependent LMI on $[0, 2]$ the numerical rank drops from 249 to 92 at degree 40 and the method stalls, whereas in a variable normalized to $[-1, 1]$ it certifies degree 40 in 19 iterations. The change of basis cannot be repaired after the program is built, so declare the variable with `nsd_timevar`, which still lets you write derivatives and endpoint values in physical time.

Prefer interval constraints to multipliers. `sosmatrixineq_interval` needs neither multipliers nor Gram variables. On a clock-dependent LMI of degree 24 it took 0.8 s, whereas the same solver with a matrix SOS multiplier did not reach a certificate in 21 s.

Ill-conditioned problems. When every solution is badly conditioned, no floating-point certificate can be trusted, whatever the solver. For a random 10-state plant with one input and sampling period 0.1, the discrete controllability Gramian has condition number about 10^{17} and every valid Lyapunov matrix about 10^{7}. The status is then `uncertified_stalled`, while the returned controller still stabilizes the sampled loop (spectral radius 0.98). For control problems, an independent check of the result (closed-loop spectral radius, second moments) is always recommended.

7 Examples

The examples in `examples/` are written one statement per line with comments, so that they can serve as templates.

7.1 Example 1: polynomial minimization

Global minimum of $x^4 - 4x^3 + 3x^2 + 2x + 1$ (0.75) and minimum of $x^4 - x$ on $[0, 2]$, the latter imposed directly on the interval:

```
prog = sosprogram(x, gam);
prog = sosineq_interval(prog, x^4 - x - gam, x, [0 2]);
prog = sossetobj(prog, -gam);
[prog, info] = sossolve_nosdp(prog);
```

Result: -0.4724704 (exact -0.4724704), certified.

7.2 Example 2: matrix SOS

The quick-start problem, in both forms of `sosmatrixineq`. Result 2.4142134 (exact $1 + \sqrt{2}$).

7.3 Example 3: sampled-data stabilization by clock-dependent LMIs

For the plant $\dot{x} = Ax + Bu$ with a zero-order hold and sampling period T, the augmented state (x, u) yields an impulsive system [1, 3]. One looks for $X(t)$, $t \in [0, T]$, and U with $\dot{X} - A_a X - X A_a^\top \succeq 0$ on $[0, T]$ and a boundary LMI, and $K = UX(T)^{-1}$ stabilizes the sampled loop.

```
[s, tm] = nsd_timevar('s', [0 T]);
prog = sosprogram(s);

[prog, X] = sospolymatrixvar(prog, monomials(s, 0:d), [m m], 'symmetric');
[prog, U] = sospolymatrixvar(prog, monomials(s, 0), [nu m]);

% flow condition on [0, T]
P = tm.ddt(X) - X*Aa' - (X*Aa')';
prog = sosmatrixineq_interval(prog, P, s, tm.interval());

% jump condition
XT = tm.at(X, T);
X0 = tm.at(X, 0);
Off = -(XT*J0' + U'*Ba')';
prog = sosmatrixineq(prog, [XT, Off'; Off, X0 - epsl*eye(m)], 'Mineq');

[prog, info] = sossolve_nosdp(prog);
```

For the unstable oscillator $A = \begin{bmatrix} 0.3 & 3 \\ -3 & 0.3 \end{bmatrix}$, $B = [0\ 1]^\top$, $T = 2$: infeasible at $d = 6$, feasible at $d = 8$ (spectral radius 0.904) and at $d = 32$ (0.539).

7.4 Example 4: largest sampling interval under aperiodic sampling

With a given gain K and sampling intervals in $[T_{\min}, T_{\max}]$, the jump condition must hold for every interval $\theta \in [T_{\min}, T_{\max}]$: it is a parameter-dependent LMI, imposed with `sosmatrixineq_interval(prog, Psi, s, tm.interval([Tmin Tmax]))`. The largest certified $T_{\max}$ is found by bisection. For the benchmark $A = \begin{bmatrix} 0 & 1 \\ 0 & -0.1 \end{bmatrix}$, $B = [0\ 0.1]^\top$, $K = -[3.75\ 11.5]$, $T_{\min} = 0.01$ and degree 8 the result is $T_{\max} = 1.7292$, against the periodic bound 1.7298 computed on the same grid.

7.5 Example 5: stochastic sampled-data systems

Theorem 15 of [2] gives conditions for the mean-square stabilization of

$$dx = (A_{sd}x + B_{sd}u)\,dt + E_{sd}x\,dW_1 + \alpha B_{sd}u\,dW_2, \qquad u(t) = K_d\,[x(t_k); u(t_k)],$$

under sampling intervals in $[T_{\min}, T_{\max}]$: a 9×9 matrix inequality on $[0, T_{\max}]$ and a 6×6 one on $[T_{\min}, T_{\max}]$, both imposed directly on their intervals (`examples/stochastic_sd_design.m`). For the paper's Example 16 all six ranges of its Table 6 are certified in under 0.5 s each. Each design is verified exactly through the second moment $E[xx^\top]$, whose evolution between and at samples is linear (`examples/second_moment.m`). For an unstable variant, bisection gives the largest certified $T_{\max}$: 1.37 at degree 2 and 3.46 at degree 4.

8 Verification

`tests/run_tests.m` contains 16 tests with known answers. They cover polynomial minima, the matrix example, half-line and interval minima, multipliers through `sossosvar` and `sossosvar_nosdp`, equality constraints, clock-dependent LMIs that are infeasible at degree 6 and feasible at degrees 8, 24, 32 and 40, Gram-free certificates, `nsd_timevar`, and a regression test ensuring that a point on the boundary of the cone is not accepted as a certificate. All pass under Octave 8.4 with SOSTOOLS 4. The workflow `.github/workflows/octave-tests.yml` runs them on every push.

With SeDuMi through SOSTOOLS' `sossolve` (`tests/compare_sedumi.m`), the optimal values of four test problems agree to about 10^{-7}. The values of SOSNOSDP lie slightly inside the feasible set, those of SeDuMi sometimes slightly outside. On the clock-dependent LMI of Example 3 at increasing degree (`tests/scaling_sedumi.m`, time interval $[0, 2]$):

degree	SeDuMi, multiplier form	SOSNOSDP, multiplier form	SOSNOSDP, native interval
8	feasible, 0.6 s	feasible, 0.7 s	feasible, 0.4 s
16	numerical error flag	feasible, 2.8 s	feasible, 0.5 s
24	numerical error	uncertified	feasible, 0.8 s

These are single runs on one core under Octave. The package has not yet been run in MATLAB by its authors.

9 Relation to other software

- **alfonso** [12] (MATLAB, Papp and Yıldız): generic non-symmetric conic solver with scalar WSOS cones in the interpolant basis, and the reference implementation of [11]. It accepts user-defined barrier oracles. https://github.com/dpapp-github/alfonso
- **Hypatia.jl** [4, 5] (Julia, Coey, Kapelevich and Vielma): generic conic interior-point solver with, among many cones, the scalar and matrix interpolant WSOS cones (`WSOSInterpNonnegativeCone`, `WSOSInterpPosSemidefTriCone`) of [6]. https://github.com/jump-dev/Hypatia.jl
- **SumOfSquares.jl** [18] (Julia, Legat et al.): JuMP-based modelling of SOS programs, the Julia counterpart of SOSTOOLS. With an interpolant (Lagrange) basis, scalar SOS constraints reach Hypatia's interpolant cone without a PSD reformulation. Matrix SOS constraints are scalarized. https://github.com/jump-dev/SumOfSquares.jl

- **SOSTOOLS** [16] (MATLAB): the modelling front end used by SOSNOSDP, which normally solves programs through SDP solvers. `https://github.com/oxfordcontrol/SOSTOOLS`

SOSNOSDP is not a port of Hypatia. Its solver is a compact MATLAB implementation of the same cone and of a Skajaa–Ye type method, and its purpose is to make the approach available to existing SOSTOOLS programs, in particular matrix-valued ones with interval constraints.

10 Limitations and future work

- Only SOSTOOLS programs written with `pvar` are supported, not the Symbolic Math Toolbox mode.
- The sparsity options of SOSTOOLS (`'sparse'`, `'sparsemultipartite'`) are handled as dense constraints.
- Global SOS constraints in several variables use approximate Fekete nodes and have been tested far less than univariate and interval constraints.
- Linear algebra is dense. The Hessian of a cone with N nodes and matrix size r has order $Nr(r+1)/2$.
- Interval constraints are univariate. Parameter boxes in several variables (robust or LPV problems) are a natural extension.
- The interior-point method uses practical parameters and has not been tuned. Certificates are evaluated in floating point, not in exact arithmetic.

11 How to cite

If you use SOSNOSDP, please cite the package (`CITATION.cff`) together with the works it builds on: Papp and Yıldız [11] for SOS optimization without SDP, Kapelevich, Coey and Vielma [6] for the matrix SOS cone and its barrier, Coey, Kapelevich and Vielma [4] for Hypatia, Skajaa and Ye [14] for the interior-point method, and SOSTOOLS [16].

References

[1] C. Briat, *Convex conditions for robust stability analysis and stabilization of linear aperiodic impulsive and sampled-data systems under dwell-time constraints*, Automatica, 49 (2013), pp. 3449–3457.

[2] C. Briat, *Stability analysis and stabilization of stochastic linear impulsive, switched and sampled-data systems under dwell-time constraints*, Automatica, 74 (2016), pp. 279–287.

[3] C. Briat, *Stability analysis and stabilization of LPV systems with jumps and (piecewise) differentiable parameters using continuous and sampled-data controllers*, Nonlinear Analysis: Hybrid Systems, 41 (2021), 101040.

[4] C. Coey, L. Kapelevich, and J. P. Vielma, *Solving natural conic formulations with Hypatia.jl*, INFORMS Journal on Computing, 34 (2022), pp. 2686–2699.

[5] C. Coey, L. Kapelevich, and J. P. Vielma, *Performance enhancements for a generic conic interior point algorithm*, arXiv:2107.04262.

[6] L. Kapelevich, C. Coey, and J. P. Vielma, *Sum of squares generalizations for conic sets*, Mathematical Programming, doi:10.1007/s10107-022-01831-6. Preprint arXiv:2103.11499.

[7] J. Löfberg and P. A. Parrilo, *From coefficients to samples: a new approach to SOS optimization*, in 43rd IEEE Conference on Decision and Control, 2004, pp. 3154–3159.

[8] Y. Nesterov, *Squared functional systems and optimization problems*, in High Performance Optimization, Kluwer, 2000, pp. 405–440.

[9] D. Papp and F. Alizadeh, *Semidefinite characterization of sum-of-squares cones in algebras*, SIAM Journal on Optimization, 23 (2013), pp. 1398–1423.

[10] D. Papp and S. Yıldız, *On "A homogeneous interior-point algorithm for non-symmetric convex conic optimization"*, arXiv:1712.00492, 2017.

[11] D. Papp and S. Yıldız, *Sum-of-squares optimization without semidefinite programming*, SIAM Journal on Optimization, 29 (2019), pp. 822–851.

[12] D. Papp and S. Yıldız, *alfonso: Matlab package for nonsymmetric conic optimization*, INFORMS Journal on Computing, 34 (2022), pp. 11–19.

[13] C. W. Scherer and C. W. J. Hol, *Matrix sum-of-squares relaxations for robust semi-definite programs*, Mathematical Programming, 107 (2006), pp. 189–211.

[14] A. Skajaa and Y. Ye, *A homogeneous interior-point algorithm for nonsymmetric convex conic optimization*, Mathematical Programming, 150 (2015), pp. 391–422.

[15] A. Sommariva and M. Vianello, *Computing approximate Fekete points by QR factorizations of Vandermonde matrices*, Computers & Mathematics with Applications, 57 (2009), pp. 1324–1336.

[16] A. Papachristodoulou, J. Anderson, G. Valmorbida, S. Prajna, P. Seiler, P. A. Parrilo, M. M. Peet, and D. Jagt, *SOSTOOLS: Sum of squares optimization toolbox for MATLAB*, version 4.00, 2021. https://github.com/oxfordcontrol/SOSTOOLS.

[17] J. F. Sturm, *Using SeDuMi 1.02, a MATLAB toolbox for optimization over symmetric cones*, Optimization Methods and Software, 11–12 (1999), pp. 625–653.

[18] B. Legat, C. Coey, R. Deits, J. Huchette, and A. Perry, *Sum-of-squares optimization in Julia*, 2017. https://github.com/jump-dev/SumOfSquares.jl.